Lab 3 — Hashing, Salting, and Digital Signatures

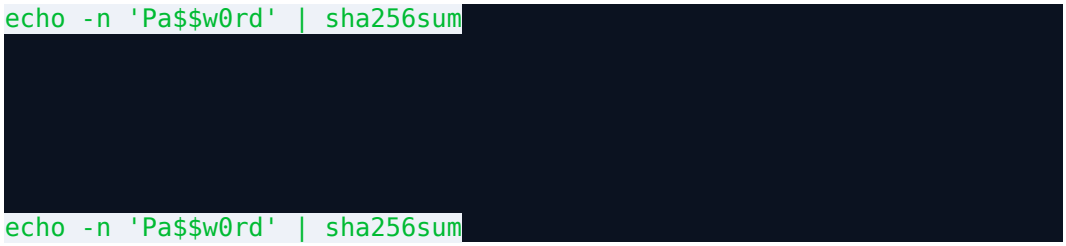
You will produce hashes, add salt, and create a digital signature with OpenSSL. This covers Security+ 1.4 (hashing, salting, digital signatures, key stretching).

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free): - `openssl` (pre-installed) - `coreutils` (pre-installed)

Step 1 — Plain hashing

```
echo -n 'Pa$$w0rd' | sha256sum
```



```
echo -n 'Pa$$w0rd' | sha256sum
```

Same input → same digest. Hashes are deterministic, which is why a **rainbow table** can crack them.

Step 2 — Salted hashing

```
SALT=$(openssl rand -hex 8)
```



```
echo "Salt: $SALT"
```

```
echo -n "${SALT}"'Pa$$w0rd' | sha256sum
```

Re-run with a different salt and observe a totally different digest. Salt defeats rainbow tables because every user has a unique salt.

Step 3 — Key stretching with bcrypt-style cost

```
openssl passwd -6 -salt $(openssl rand -hex 4) 'Pa$$w0rd'
```

`-6` selects SHA-512 with 5000 rounds (the format Linux `/etc/shadow` uses). The repeated rounds are **key stretching** — they make brute force expensive.

Step 4 — Generate an RSA key pair

```
openssl genrsa -out priv.pem 2048
openssl rsa -in priv.pem -pubout -out pub.pem
ls -l priv.pem pub.pem
```

Step 5 — Sign and verify a document

```
echo "Transfer 1000 USD to Bob" > order.txt
```

```
openssl dgst -sha256 -sign priv.pem -out order.sig order.txt
```

```
openssl dgst -sha256 -verify pub.pem -signature order.sig order.txt
```

Output `Verified OK` proves: (1) the document was not modified (**integrity**) and (2) it could only be signed by the holder of `priv.pem` (**non-repudiation**).

Tamper and re-verify:

```
echo " - extra line" >> order.txt
```

```
openssl dgst -sha256 -verify pub.pem -signature order.sig order.txt
```

You should see `Verification Failure`.

Free online tools

- **Hash Generator** — SHA-2, MD5, HMAC in the browser: https://alfredang.github.io/hashgenerator/
- **Cryptography Toolkit** — AES, RSA, signing/verification: https://alfredang.github.io/cryptography-toolkit/
- **Online RSA Key Generator** — https://travistidwell.com/jsencrypt/demo/

What you learned

- Why salt is mandatory for password storage.
- What "key stretching" buys you against offline brute force.
- How a signature delivers integrity **and** non-repudiation in one step.